

Rapport final RLMinimax

Projet Pac-Man ALG4 - rapport personnel de verification et de defense

Je presente mon correctif final. Mon agent RLMinimax combine une recherche Alpha-Beta avec une fonction d evaluation affine. Les poids de cette fonction sont appris par reinforcement learning, puis utilises pour evaluer les positions pendant la recherche.

Conformite Alpha-Beta + fonction affine + poids appris.	Resultats RL meilleur sur 10/10 layouts testes.	Verification Syntaxe, poids et formule controles.
---	---	---

1. Conformite au sujet

Point	Ce que j ai implemente	OK
Facteurs Pac-Man	7 features : nourriture, fantomes, capsules, score, nourriture restante	OK
Recherche	BFS pour calculer des distances reelles dans le labyrinthe	OK
Fonction affine	evaluation = somme des ai*xi + constante C	OK
Alpha-Beta	Pac-Man MAX, fantomes MIN, coupure alpha >= beta	OK
Poids appris	train.py ajuste les poids par apprentissage	OK
Comparaison	compare.py mesure score, winrate, temps, coups et Stop	OK

2. Fonction affine et poids actuels

Modele utilise : $f(s) = a1 \cdot x1(s) + a2 \cdot x2(s) + \dots + a7 \cdot x7(s) + C$

Fact.	Fonction	Poids	Information	Sens
x1	nearest_food_bfs	0.6891	Nourriture proche	Chercher nourriture
x2	ghosts_within_3	-1.4093	Fantomes dangereux	Eviter danger
x3	scared_ghosts_nearby	1.4061	Fantomes effrayes	Profiter opportunités
x4	remaining_food	0.4036	Nourriture restante	Finir la carte
x5	nearest_capsule_bfs	0.0252	Capsule proche	Effet positif faible
x6	current_score	1.8572	Score normalise	Score important
x7	nearest_dangerous_ghost_bfs	-1.2624	Danger principal	Rester loin
C	bias	-0.0834	Correction globale	Constante

Ce que je dois expliquer a l oral

- Le reinforcement learning ne remplace pas Alpha-Beta : il apprend uniquement les poids de la fonction d evaluation.
- La fonction respecte le modele affine demande : pas de termes croises, pas de carres, pas de division par une feature.
- Les poids sont stockes dans weights.json et correspondent exactement aux 7 features de features.py.
- La comparaison utilise seed=0 pour garder des conditions identiques entre les agents.

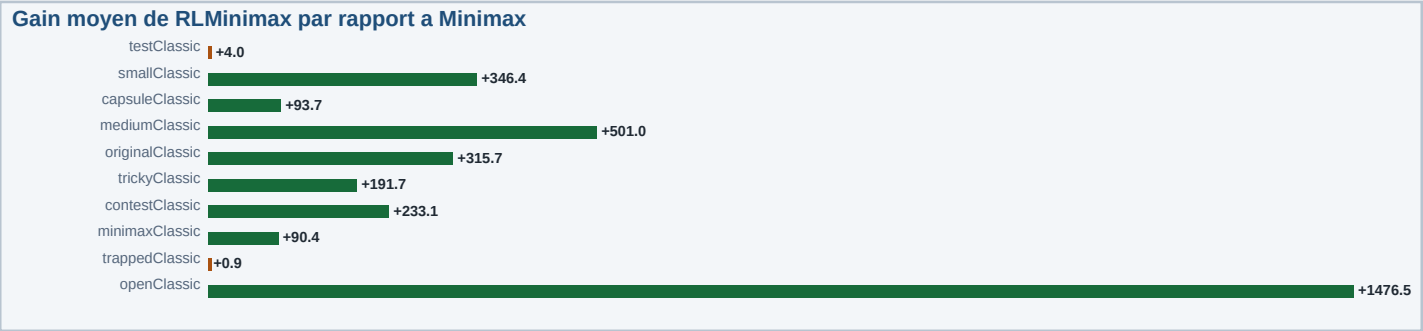
Resultats, analyse et conclusion

Tests relances le 14/05/2026 avec les poids actuels, sans reentrainement

3. Resultats finaux

Layout	D	N	Minimax	AlphaBeta	RL	Diff	Win
testClassic	1	200	548.5	548.5	552.5	+4.0	100%
smallClassic	2	100	-149.8	-149.8	196.6	+346.4	34%
capsuleClassic	2	100	-228.9	-228.9	-135.2	+93.7	12%
mediumClassic	1	50	14.5	14.5	515.5	+501.0	28%
originalClassic	1	50	348.6	348.6	664.4	+315.7	2%
trickyClassic	1	50	101.3	101.3	293.0	+191.7	2%
contestClassic	1	50	11.3	11.3	244.4	+233.1	28%
minimaxClassic	3	100	-243.5	-243.5	-153.1	+90.4	34%
trappedClassic	1	1000	-392.3	-392.3	-391.4	+0.9	11%
openClassic	1	10	-419.5	-745.4	1057.0	+1476.5	90%

Lecture : Diff est positive partout, donc RLMinimax obtient un meilleur score moyen sur tous les layouts testes.



4. Analyse et justification

Lecture honnete

- Le winrate n est pas parfait sur toutes les cartes, mais le score moyen est meilleur.
- openClassic est particulier : les baselines dépassent le timeout choisi.
- Le projet reste conforme : fonction affine, poids appris et comparaison statistique.

Pourquoi c est logique

- BFS donne une distance réelle dans le labyrinthe.
- Alpha-Beta garde la logique Minimax en coupant des branches inutiles.
- Les poids viennent d un entrainement, pas d un choix manuel.

Conclusion finale

Le projet respecte le sujet : fonction affine, facteurs Pac-Man, BFS, Alpha-Beta, poids appris, resultats statistiques et justification claire. Le rapport est pret pour la partie technique ; il reste seulement a verifier le depot Git/Gitesi avant la remise.

Phrase courte pour l oral : mon agent utilise Alpha-Beta pour explorer les coups, puis une fonction affine apprise par renforcement pour evaluer les positions.